Loop Gestión

Documento de Arquitectura y Diseño (SDD)

Sebastián Grande Moreno y Juan Carlos Díaz Cifuentes

Docente: MGTR. Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura, Bogotá · Optativa II (10348) — Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

Tabla de contenido

Información del documento 3

1. Decisiones de arquitectura 4

2. Vista de componentes 5

3. Modelo de datos (conceptual) 6

4. Vista dinámica: publicar una actividad 8

5. Navegación 9

6. Contrato de la API (referencia de diseño) 10

7. Integración continua 11

Información del documento

Campo	Valor
Proyecto	Loop Gestión — Plataforma académica full-stack con automatización e IA
Documento	SDD — Arquitectura y Diseño
Estándar de referencia	IEEE 1016-2009
Versión	1.0
Autores	Sebastián Grande Moreno y Juan Carlos Díaz Cifuentes
Asignatura	Optativa II (10348) — Desarrollo de Aplicaciones para Dispositivos Móviles
Docente	MGTR. Ing. Jairo Armando Salcedo Aranda
Institución	Universidad de San Buenaventura, Bogotá
Fecha	Noviembre de 2026

> **Nota sobre el alcance de este documento.** Se elaboró a partir del proyecto en Azure DevOps (backlog, sprints, tablero, repositorio y pipeline, corte del 4 de noviembre de 2026) y de los mockups de la aplicación. El código fuente se encuentra en el repositorio de Azure Repos del equipo. Los datos que dependen directamente del código (nombres exactos de rutas, tablas, variables y versiones) están marcados con ⚠️ **Por confirmar con el código** para validarlos antes de la entrega final.

1. Decisiones de arquitectura

Decisión	Alternativa considerada	Justificación
Cliente-servidor con API REST	App conectada directo a la base de datos	La base de datos no se expone a internet; las reglas de negocio quedan en un solo lugar.
Node.js + Express	Firebase Functions / Django	Ligero, mismo lenguaje (JavaScript) del pipeline y de las pruebas (chatbot.test.js).
MySQL en Docker	MySQL instalado localmente	Misma versión en todos los equipos; se levanta con un comando.
Firebase Authentication	Contraseñas propias en MySQL	Google gestiona contraseñas, recuperación y seguridad de las cuentas.
Retrofit en Android	HttpURLConnection	Convierte JSON a objetos Kotlin y simplifica las peticiones.
Make.com para notificaciones	Enviar correos desde la API	Integra Gmail y Google Sheets sin programar SMTP ni la API de Sheets.

2. Vista de componentes

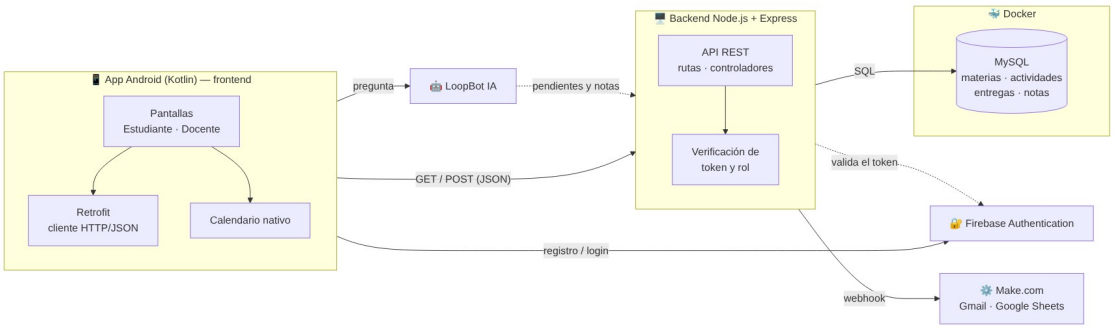


Figura 1. Componentes y comunicación

Componente	Responsabilidad
Pantallas (Activities / Compose) ⚠️ Por confirmar con el código	Mostrar la información y capturar acciones del estudiante o docente.
Repositorios y servicios Retrofit	Llamar a la API, convertir JSON a modelos Kotlin y manejar errores de red.
API REST (rutas y controladores)	Recibir peticiones, validar datos y aplicar reglas de negocio.
Middleware de autenticación	Verificar el token de Firebase y el rol del usuario en cada petición.
Capa de datos (MySQL)	Persistir materias, matrículas, actividades, entregas y notas.
Integración Make.com	Enviar un webhook al publicar actividades.
LoopBot IA	Responder preguntas usando los datos del estudiante que entrega la API.

3. Modelo de datos (conceptual)

El siguiente modelo muestra las entidades del dominio. Los nombres exactos de tablas y columnas deben confirmarse con los scripts SQL del repositorio (⚠️ Por confirmar con el código).

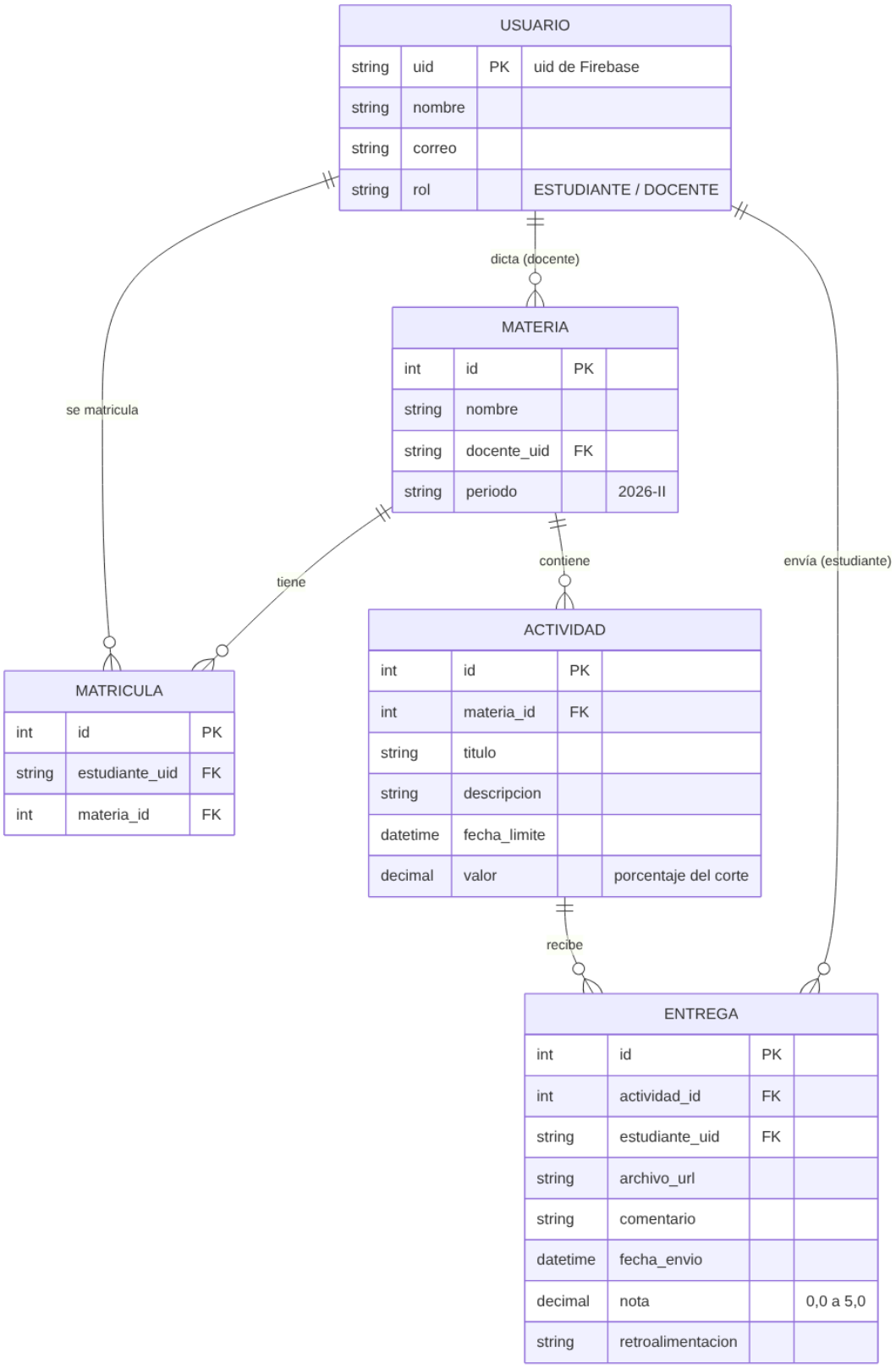


Figura 2. Modelo entidad-relación conceptual

4. Vista dinámica: publicar una actividad

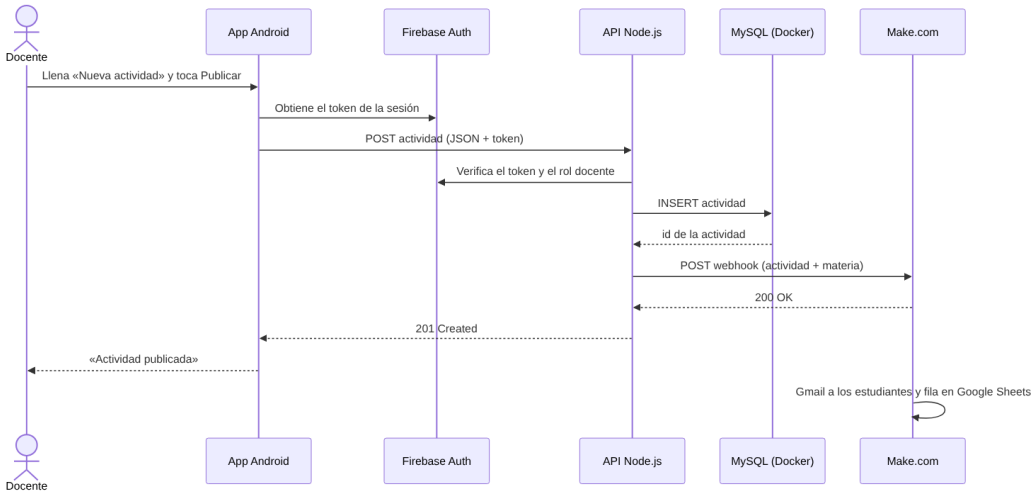


Figura 3. Secuencia al publicar una actividad con notificación

5. Navegación

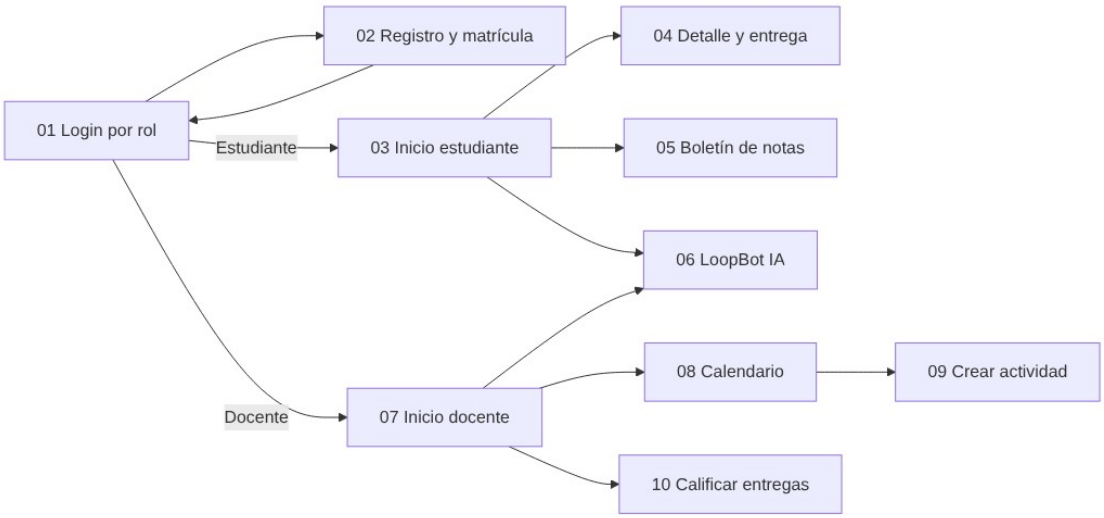


Figura 4. Mapa de navegación de la app

6. Contrato de la API (referencia de diseño)

Tabla de referencia de los recursos que la app necesita. Verificar las rutas reales en el código del backend (⚠️ Por confirmar con el código).

Método	Ruta (referencia)	Uso	Rol
POST	/api/usuarios	Registrar usuario y matrícula después del alta en Firebase	Todos
GET	/api/materias	Materias del usuario autenticado	Todos
GET	/api/actividades?materia={{id}}	Actividades de una materia	Todos
POST	/api/actividades	Crear actividad (dispara el webhook)	Docente
POST	/api/entregas	Enviar entrega	Estudiante
PUT	/api/entregas/{{id}}/calificacion	Calificar entrega	Docente
GET	/api/notas	Boletín del estudiante autenticado	Estudiante
POST	/api/chatbot	Pregunta a LoopBot	Estudiante

Códigos de respuesta esperados: 200 OK, 201 Created, 400 datos inválidos, 401 sin sesión, 403 rol no autorizado, 404 no encontrado, 500 error del servidor.

7. Integración continua

El repositorio contiene azure-pipelines.yml. La ejecución #20261104.1 sobre la rama develop terminó en verde (43 s) y reportó 3 advertencias: migración de la imagen ubuntu-latest y la tarea deprecada NodeTool@0, que debe reemplazarse por UseNode@1. El repositorio está organizado en docs/, src/ y tests/, y el PR 1 «Integración de pruebas de LoopBot IA y estructura del repositorio» se integró a main desde feature/chatbot-loopbot.